

CONTROL STRUCTURES

Scenario 1: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
  FOR c IN (
```

```
    SELECT CustomerID, DOB
```

```
    FROM Customers
```

```
  )
```

```
  LOOP
```

```
    IF TRUNC(MONTHS_BETWEEN(SYSDATE, c.DOB) / 12) > 60 THEN
```

```
      UPDATE Loans
```

```
      SET InterestRate = InterestRate - 1
```

```
      WHERE CustomerID = c.CustomerID;
```

```
      DBMS_OUTPUT.PUT_LINE('Discount applied for Customer ID: ' ||  
c.CustomerID);
```

```
    END IF;
```

```
  END LOOP;
```

```
  COMMIT;
```

```
END;
```

```
/
```

Output:

The screenshot shows the FreeSQL web interface. On the left, the 'Navigator' pane displays a schema named 'My Schema' with a table named 'Tables'. Below this, a search bar and a list of tables (ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, TRANSACTIONS) are visible. The main area shows a PL/SQL script in a worksheet. The script is as follows:

```
1 SET SERVEROUTPUT ON;
2
3 BEGIN
4   FOR c IN (
5     SELECT CustomerID, DOB
6     FROM Customers
7   )
8   LOOP
9     IF TRUNC(MONTHS_BETWEEN(SYSDATE, c.DOB) / 12) > 60 THEN
10      UPDATE Loans
11      SET InterestRate = InterestRate - 1
12      WHERE CustomerID = c.CustomerID;
13      DBMS_OUTPUT.PUT_LINE('Discount applied for Customer ID: ' || c.CustomerID);
14    END IF;
15  END LOOP;
16 COMMIT;
17 END;
```

Below the script, the 'Script output' tab is selected, showing the following results:

```
Discount applied for Customer ID: 3
Discount applied for Customer ID: 5

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.177
```

Scenario 2: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

```
ALTER TABLE Customers
ADD IsVIP CHAR(1);
```

```
SET SERVEROUTPUT ON;
```

```
BEGIN
  FOR c IN (
    SELECT CustomerID, Balance
    FROM Customers
  )
  LOOP
    IF c.Balance > 10000 THEN
      UPDATE Customers
      SET IsVIP = 'Y'
      WHERE CustomerID = c.CustomerID;
```

```

        DBMS_OUTPUT.PUT_LINE('Customer ' || c.CustomerID || ' promoted to VIP.');
```

ELSE

```

        UPDATE Customers
        SET IsVIP = 'N'
        WHERE CustomerID = c.CustomerID;
    END IF;
END LOOP;

COMMIT;
END;
/
```

Output:

The screenshot shows the FreeSQL online SQL editor interface. On the left is a 'Navigator' pane with a tree view showing 'My Schema' and 'Tables'. Below this is a search bar labeled 'Search objects'. The main area displays a SQL script in a text editor with line numbers 1 through 22. The script is a PL/SQL block that iterates over customers and promotes those with a balance greater than 10,000 to VIP status. Below the script editor are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is active, showing the results of the script execution: 'Customer 2 promoted to VIP.', 'Customer 3 promoted to VIP.', and 'Customer 5 promoted to VIP.'. At the bottom, a status message reads 'PL/SQL procedure successfully completed.'

```

1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      FOR c IN (
5          SELECT CustomerID, Balance
6          FROM Customers
7      )
8      LOOP
9          IF c.Balance > 10000 THEN
10             UPDATE Customers
11             SET IsVIP = 'Y'
12             WHERE CustomerID = c.CustomerID;
13
14             DBMS_OUTPUT.PUT_LINE('Customer ' || c.CustomerID || ' promoted to VIP.');
```

ELSE

```

16         UPDATE Customers
17         SET IsVIP = 'N'
18         WHERE CustomerID = c.CustomerID;
19     END IF;
20 END LOOP;
21
22 COMMIT;
```

Customer 2 promoted to VIP.
Customer 3 promoted to VIP.
Customer 5 promoted to VIP.

PL/SQL procedure successfully completed.

Scenario 3: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
SET SERVEROUTPUT ON;
```

```

BEGIN
  FOR I IN (
    SELECT c.Name,
           I.LoanID,
           I.EndDate
    FROM Customers c
    JOIN Loans I
    ON c.CustomerID = I.CustomerID
    WHERE I.EndDate BETWEEN SYSDATE AND SYSDATE + 30
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Reminder: Loan ID '
      || I.LoanID
      || ' for '
      || I.Name
      || ' is due on '
      || TO_CHAR(I.EndDate,'DD-MON-YYYY')
    );
  END LOOP;
END;
/

```

Output:

My Schema

Tables

Search objects

- ACCOUNTS
- CUSTOMERS
- EMPLOYEES
- LOANS
- TRANSACTIONS

[SQL Worksheet]*

```
1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      FOR l IN (
5          SELECT c.Name,
6                 l.LoanID,
7                 l.EndDate
8          FROM Customers c
9          JOIN Loans l
10         ON c.CustomerID = l.CustomerID
11         WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
12      )
13  LOOP
14      DBMS_OUTPUT.PUT_LINE(
15          'Reminder: Loan ID '
16          || l.LoanID
17          || ' for '
18          || l.Name
19          || ' is due on '
20          || TO_CHAR(l.EndDate, 'DD-MON-YYYY')
21      );
22  END LOOP;
```

Query result Script output DBMS output Explain Plan SQL



Reminder: Loan ID 203 for Robert Wilson is due on 01-AUG-2026
Reminder: Loan ID 204 for Emily Davis is due on 27-JUL-2026

PL/SQL procedure successfully completed.